

EBAC

PROJETO 4

Plataforma de Streaming

Banco de Dados

versão 1.1

1.1- Escolha do Banco de Dados

O banco de dados escolhido foi o MySQL devido a minha experiência do uso com o MySQL Workbench 8.0.

1.2- Dados sobre o funcionamento

Para estruturar o banco de dados, o diagrama de entidade-relacionamento (DER), aplicado a nomenclatura das tabelas, colunas e relacionamentos será diretamente em inglês, garantindo a padronização do mercado e as boas práticas para o código a ser desenvolvido.

Com base nas regras de negócio e requisitos técnicos fornecidos, o sistema exige três entidades mínimas principais: **Usuários**, **Filmes** e **Histórico**.

1.3- Detalhamento das Entidades

1.3.1-Users (Usuários): esta entidade armazenará tanto os usuários assinantes quanto os não assinantes.

A coluna `subscriber_id` é um autorrelacionamento (Foreign Key apontando para a própria tabela `users`), garantindo que usuários não assinantes compartilhem a conta e fiquem vinculados ao assinante principal.

A coluna `age` armazenará a idade informada, que servirá de base para determinar o `profile_type` (Infantil, Adolescente ou Adulto) no momento da criação.

A coluna `allows_adult_content` permite que o assinante conceda permissão específica para perfis adolescentes acessarem filmes de classificação 18 anos.

1.3.2-Movies (Filmes): entidade que armazena o catálogo disponível.

Contém atributos obrigatórios como o `title` e a classificação etária (`age_rating`), que pode ser 0, 12, 16 ou 18 anos.

A coluna `minimum_plan` define qual o plano mínimo (Basic, Standard ou Premium) exigido para que o filme seja acessado.

1.3.3-Movie_genres (Gêneros dos Filmes): Como um filme pode ter múltiplos gêneros cadastrados ao mesmo tempo, foi extraída essa informação para uma tabela separada com relação de N-para-1 com a tabela de filmes.

1.3.4-Watch_histories (Históricos de Filmes Assistidos): Sempre que a API for acionada para assistir a um filme, um novo registro será inserido.

A coluna `watched_at` garante que a listagem mantenha a ordem cronológica em que os filmes foram assistidos, além de permitir o registro de múltiplas repetições do mesmo filme pelo mesmo usuário.

1.4- EER-diagrama via MySQL

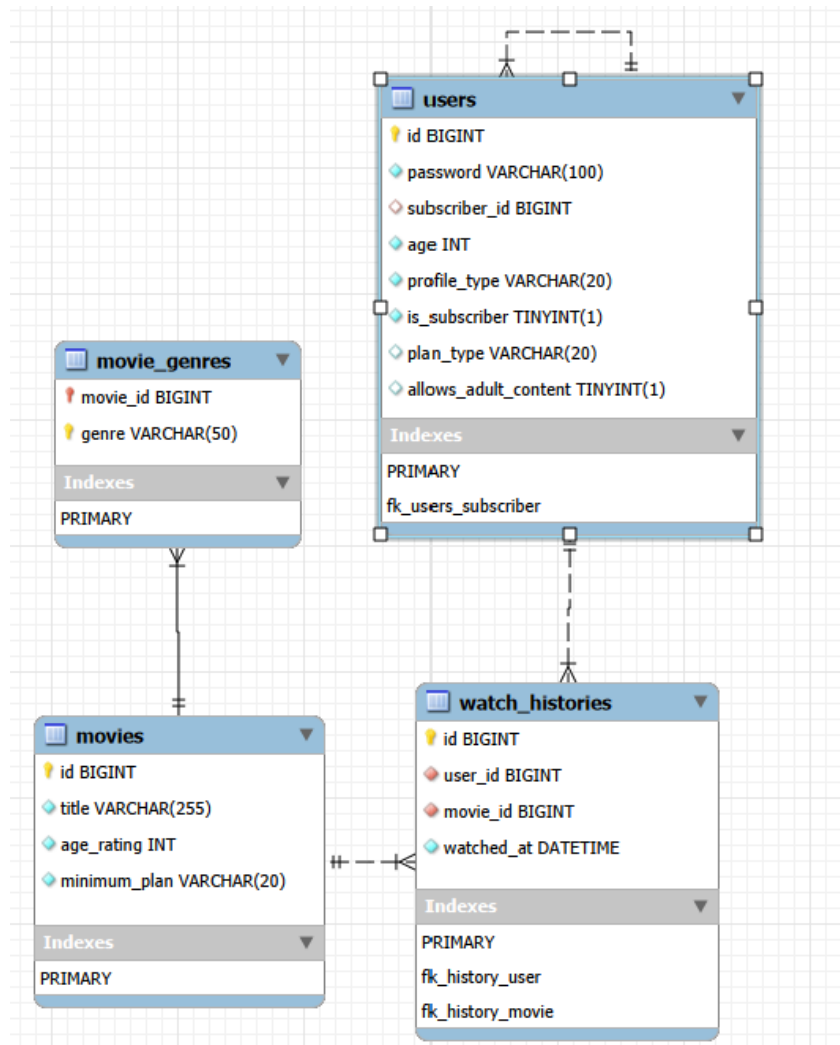


Imagem 1: EER Diagram

1.5- Script de Criação do Banco de Dados (MySQL)

```
-- Cria o banco de dados se ele não existir e define o charset padrão
CREATE DATABASE IF NOT EXISTS streaming_platform
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;

-- Seleciona o banco de dados para os próximos comandos
USE streaming_platform;
```

```

-- 1. Tabela de Usuários (Users)
CREATE TABLE users (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  password VARCHAR(100) NOT NULL,
  subscriber_id BIGINT,
  age INT NOT NULL,
  profile_type VARCHAR(20) NOT NULL,      -- Ex: 'CHILD', 'TEEN', 'ADULT'
  is_subscriber BOOLEAN NOT NULL,
  plan_type VARCHAR(20),                  -- Ex: 'BASIC', 'STANDARD', 'PREMIUM'
  (Pode ser NULL para não assinantes)
  allows_adult_content BOOLEAN DEFAULT FALSE,
  -- Autorrelacionamento: Se a conta do assinante for deletada, os perfis vinculados
  também serão
  CONSTRAINT fk_users_subscriber FOREIGN KEY (subscriber_id) REFERENCES
  users(id) ON DELETE CASCADE
);

-- 2. Tabela de Filmes (Movies)
CREATE TABLE movies (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  title VARCHAR(255) NOT NULL,
  age_rating INT NOT NULL,                -- Valores esperados: 0, 12, 16, 18
  minimum_plan VARCHAR(20) NOT NULL      -- Ex: 'BASIC', 'STANDARD',
  'PREMIUM'
);

-- 3. Tabela de Gêneros dos Filmes (Movie Genres)
CREATE TABLE movie_genres (
  movie_id BIGINT NOT NULL,
  genre VARCHAR(50) NOT NULL,
  -- Chave primária composta para evitar gêneros duplicados no mesmo filme
  PRIMARY KEY (movie_id, genre),
  -- Se o filme for deletado, seus gêneros também serão
  CONSTRAINT fk_movie_genres_movie FOREIGN KEY (movie_id) REFERENCES
  movies(id) ON DELETE CASCADE
);

-- 4. Tabela de Histórico (Watch Histories)
CREATE TABLE watch_histories (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id BIGINT NOT NULL,
  movie_id BIGINT NOT NULL,
  watched_at DATETIME NOT NULL,
  -- Relacionamentos com deleção em cascata para manter a integridade referencial
  CONSTRAINT fk_history_user FOREIGN KEY (user_id) REFERENCES users(id)
  ON DELETE CASCADE,
  CONSTRAINT fk_history_movie FOREIGN KEY (movie_id) REFERENCES
  movies(id) ON DELETE CASCADE
);

```

Informações:

- **ON DELETE CASCADE no subscriber_id**
 - Atendendo à regra de negócio: "se um usuário assinante excluir sua conta, todos os usuários vinculados a ele devem ser excluídos junto com ele". O próprio banco de dados já garante essa integridade, poupando processamento no back-end.
- **Chave Primária Composta (movie_id, genre)**
 - Na tabela movie_genres, a combinação do ID do filme com a String do gênero forma a chave primária. Isso impede o cadastro acidental de uma mesma gênero para o mesmo filme.
- **Charset e Collation**
 - Adicionado o utf8mb4 na criação do banco para garantir que a aplicação suporte todo tipo de caractere especial e acentuação nos títulos dos filmes sem problemas de codificação.